Quando orquestrar agentes não compensa:
o que a pesquisa mostra e o que medimos no NarraTer

Gabriel Cortez Claude Code

16 de outubro de 2026

Resumo

O NarraTer existe para pôr vários agentes de IA para trabalhar juntos. Este documento faz a pergunta incômoda: na maior parte das tarefas de programação, isso compensa? Reunimos o que artigos e equipes de produção mediram em 2025 e 2026 e o comparamos com o nosso próprio benchmark. A resposta da literatura é consistente. Quando um agente sozinho já resolve bem a tarefa, uma equipe custa de 1,8 a 5,1 vezes mais e quase nunca melhora o resultado de forma significativa. Em tarefas sequenciais, ela piora. A equipe compensa em casos estreitos: trabalho realmente paralelo, contexto que não cabe num agente só e verificação independente. Medimos isso no NarraTer com o mesmo pedido dado a um terminal sozinho e à equipe de três agentes (seção 3). Os dois tiraram 31 de 31 nos testes escondidos. O terminal sozinho foi 2,5 vezes mais rápido (1,6 contra 3,9 min), usou 7 vezes menos tokens e custou 26% menos que a versão atual da equipe (G).

Rótulos. [citado]: afirmado pela fonte indicada. [medido]: medido por nós, com método reproduzível. [inferido]: nossa leitura, sem medição direta.

1 A pergunta

Montar uma equipe de agentes parece óbvio: um líder divide a tarefa, especialistas executam em paralelo e o resultado sai mais rápido e melhor. Nosso benchmark de versões (relatório *orquestracao-versoes.pdf*) mediu o *custo* de orquestrar: tokens, tempo e falhas do protocolo. Ele não mediu se orquestrar *vale a pena*. Todas as rodadas usaram a equipe, e nenhuma o agente sozinho. Este documento responde a essa segunda pergunta, primeiro com a literatura e depois com uma medição nossa.

2 O que a pesquisa mediu

2.1 Com o mesmo orçamento, um agente sozinho empata ou ganha

A comparação justa é a de mesmo orçamento: a equipe gasta mais tokens, então precisa ser comparada com um agente sozinho que também pode gastar mais (pensar mais, tentar de novo).

- [citado] Kim et al. [1] avaliaram 180 configurações de sistemas de agentes. A coordenação dá "retorno decrescente ou negativo quando o agente sozinho passa de ∼45%" de acerto. Em tarefas paralelizáveis, a coordenação centralizada ganha 80,8%. Em raciocínio sequencial, as variantes multiagente perdem de 39% a 70%.
- [citado] Tran e Kiela [2] mostram que, com o mesmo orçamento de tokens de raciocínio, um agente sozinho empata ou supera sistemas multiagente em raciocínio de vários passos.
- [citado] Wang et al. [3] chegam ao mesmo resultado: um agente sozinho bem instruído chega perto da melhor discussão entre agentes. A discussão só vence quando não há exemplos no prompt.

- [citado] Zhang et al. [4] (5 métodos de debate, 9 benchmarks) concluem que o debate entre agentes raramente supera o raciocínio encadeado ou a autoconsistência de um agente só. A exceção é usar modelos diferentes entre si.

2.2 Em programação, equipes custam mais e quase não melhoram

A Vals.ai publicou em 9 de outubro de 2026 o estudo mais próximo do nosso caso [5]. A tarefa era construir aplicações web inteiras, com nota dada por testes de interface. Cada modelo rodou sozinho e como equipe (um líder e até cinco subagentes), em dois níveis de esforço.

Tabela 1: Agente sozinho contra equipe na Vibe Code Bench [citado] [5].

Modelo (esforço)	Sozinho	Equipe	Custo da equipe	Tempo mediano (min)
GPT 6 Sol (médio)	77,6%	84,9% (+7,3*)	2,5×	12,4 → 11,9
GPT 6 Sol (máximo)	89,0%	90,4% (+1,4)	1,8×	38,4 → 27,6
Opus 5.5 (médio)	91,5%	91,2% (−0,3)	2,2×	20,3 → 26,6
Opus 5.5 (máximo)	89,8%	93,2% (+3,4)	5,1×	74,7 → 174,1

* a única diferença estatisticamente significativa ($p = 0{,}005$).

Das quatro comparações, só uma teve ganho significativo, e foi justamente no modelo com a nota mais baixa sozinho. Com o Opus 5.5 no esforço médio, a configuração mais parecida com o nosso Narrador, a equipe ficou 0,3 ponto abaixo, custando 2,2 vezes mais e levando 6 minutos a mais. No esforço máximo, a equipe do Opus custou US$ 122 por aplicação, contra US$ 23,77 sozinho, e levou 174 minutos contra 75.

Outras medições em código apontam na mesma direção:

- [citado] O Agentless [6] é um pipeline fixo (localizar, corrigir, validar), sem nenhum agente planejando. Resolveu 32% do SWE-bench Lite a US$ 0,70 por problema, competitivo com sistemas de agentes muito mais caros na época.
- [citado] A Anthropic, descrevendo o próprio sistema multiagente de pesquisa [7], diz que programação tem "menos tarefas realmente paralelizáveis que pesquisa" e que multiagente encaixa mal onde todos precisam do mesmo contexto.
- [citado] Um estudo de 2026 sobre topologias em código do zero [8] mostra que só mudar a topologia move a nota em mais de 30 pontos e dobra o tempo. Pipelines estruturados convergem mais rápido e com mais qualidade, e "supervisão gerencial pesada piora o desempenho".

2.3 Por que a equipe custa tanto

O sistema de pesquisa da Anthropic gasta cerca de 15 vezes os tokens de uma conversa, contra cerca de 4 vezes de um agente sozinho [7] [citado]. Lá, a variação de tokens sozinha explica 80% da variação de desempenho. Isso significa que boa parte do ganho de uma equipe é simplesmente gastar mais. Os mecanismos que a literatura descreve:

1. **Tradução entre agentes.** Cada delegação reescreve a tarefa, e cada resposta é lida e resumida de novo pelo líder. No benchmark da LangChain [9], a maior parte dos erros do supervisor vinha desse "telefone sem fio". Repassar a resposta palavra por palavra melhorou o resultado em quase 50% [citado].
2. **Decisões implícitas em conflito.** A Cognition [10] mostra subagentes paralelos produzindo peças incompatíveis: "ações carregam decisões implícitas, e decisões em conflito geram resultados ruins". Em 2026 [11], a mesma equipe conclui que o multiagente funciona quando "a escrita fica em uma linha só" e os outros agentes contribuem inteligência, não ações [citado].
3. **Amplificação de erro.** Agentes independentes amplificam erros 17,2 vezes; a coordenação centralizada contém isso em 4,4 vezes [1] [citado].

4. **Verificação fraca.** Na taxonomia MAST [12], tirada de 150 traços anotados de 7 frameworks, cerca de 44% das falhas são de projeto do sistema (repetir passos 15,7%, não saber quando parar 12,4%), cerca de 32% de desalinhamento entre agentes e cerca de 23,5% de verificação (verificação errada 9,1%, ausente 8,2%, término prematuro 6,2%). Em trajetórias de código, 75,8% das falhas com declaração de status eram um "pronto" falso, e juízes LLM não passaram de AUROC 0,65 para detectá-lo [13] [citado].

2.4 Quando a equipe compensa

A mesma literatura aponta os casos em que vale montar uma equipe:

- **Trabalho realmente paralelo.** Subtarefas independentes, de preferência de leitura (pesquisar, comparar, auditar), com coordenação centralizada: +80,8% [1]. A pesquisa da Anthropic venceu o Opus sozinho por 90,2% numa avaliação interna [7] [citado].
- **Contexto que não cabe num agente.** Com muitas ferramentas ou domínios misturados, um agente sozinho "cai bruscamente" a partir de dois domínios de distração [9] [citado].
- **Verificação independente.** Um revisor com contexto limpo, que vê só o diff, acha em média 2 bugs por PR, cerca de 58% deles graves [11]. Rodar os testes como retorno rendeu de 4 a 5 pontos no MetaGPT [14] [citado].
- **Modelo base fraco na tarefa.** O único ganho significativo da Vals.ai foi no modelo com a menor nota sozinho [5], coerente com o limiar de ∼45% de Kim et al. [inferido]

3 O que medimos no NarraTer

3.1 O que o estudo de versões já dizia

No estudo de versões (A a G, 3 rodadas cada), **todas** as rodadas tiraram 31 de 31 nos testes escondidos [medido]. Pelo critério de Kim et al., a tarefa está muito acima do limiar de 45%: só custo e tempo podem diferir. O custo cresceu com o número de pedidos que o Narrador fazia aos devs, e a latência do protocolo em si ficou em segundos. Na melhor versão (G), a equipe gastou 3,9 min e US$ 0,74 por rodada, em média [medido].

3.2 O braço "sozinho"

Método. Usamos o mesmo benchmark, a mesma tarefa (o repositório `pedidos`, com uma funcionalidade de várias moedas e dois defeitos plantados) e os mesmos 31 testes escondidos. Cada rodada começa do zero: HOME, tmux e sessão próprios. Há uma diferença só: o canvas tem **um terminal**, com o mesmo Opus 5.5 do Narrador e o papel de desenvolvedor, porque o papel de líder proíbe editar. O pedido é o mesmo, trocando "distribua o trabalho entre DevA e DevB" por "você trabalha sozinho". O critério de pronto e a linha de fim não mudam. Foram 3 rodadas, na versão atual (H, `73fab8b`), em 16/10/2026 (`BENCH_VARIANTE=solo python3 rodar.py H N`; comparação em `bench/orquestracao/solo.py`). Para a equipe, usamos as rodadas F e G do estudo de versões, as duas melhores. A H só acrescenta o painel Activity à G, sem mudar o protocolo.

Tabela 2: Um terminal sozinho contra a equipe de três agentes, na mesma tarefa [medido]. Média e, entre parênteses, o mínimo e o máximo das 3 rodadas. A equipe F tem o mesmo protocolo de leitura do transcript da G, sem o ajuste de pedidos em lote.

	Sozinho (H)	Equipe (G)	Equipe (F)
Testes escondidos (de 31)	31 (31–31)	31 (31–31)	31 (31–31)
Duração (min)	1,6 (1,4–1,8)	3,9 (2,8–5,2)	2,8 (2,3–3,7)
Tokens processados (milhões)	0,36 (0,35–0,36)	2,50 (2,01–2,85)	1,73 (1,27–2,28)
Custo de API equivalente (US$)	0,55 (0,52–0,58)	0,74 (0,63–0,88)	0,65 (0,50–0,82)
Chamadas à API (todos os agentes)	8 (8–8)	49 (42–56)	38 (30–49)
Pedidos do Narrador a outros	0 (0–0)	4 (3–6)	4 (3–7)
Linhas no diff	368 (340–412)	454 (413–478)	394 (316–524)

Resultado. [medido]

- **Mesma qualidade.** As seis rodadas da equipe e as três rodadas sozinho tiraram 31 de 31. Os testes do próprio agente também passaram todos (40 a 47).
- **2,5 vezes mais rápido que a equipe G**: 1,6 min contra 3,9 min, sem sobreposição (Mann–Whitney $U = 0$, $p = 0{,}10$, o menor p possível com 3 contra 3). Contra a F, a melhor equipe em tempo, o agente sozinho também vence sem sobreposição (1,6 contra 2,8 min, $p = 0{,}10$).
- **7 vezes menos tokens** (0,36 contra 2,50 milhões) e **6 vezes menos chamadas à API** (8 contra 49), também sem sobreposição. As três rodadas sozinho fizeram exatamente 8 chamadas.
- **26% mais barato que a G** (US$ 0,55 contra 0,74, sem sobreposição, $p = 0{,}10$). Contra a F, o custo se sobrepõe (US$ 0,50 a 0,82; $p = 0{,}70$): não dá para afirmar diferença.

Por que o custo cai menos que os tokens. Na equipe, quem processa mais tokens é o DevB, o Haiku: de 1,3 a 1,7 milhão de tokens por rodada na G, a menos de US$ 0,04. O agente sozinho é o Opus, o modelo mais caro. Mesmo assim, **só o Narrador da equipe G, apenas coordenando, custou de US$ 0,32 a 0,41 por rodada**: de 60% a 75% do que o Opus sozinho gastou para fazer a tarefa inteira [medido]. Coordenar sai quase tão caro quanto fazer.

Leitura. [inferido] É o que Kim et al. [1] e a Vals.ai [5] preveem para uma tarefa que o agente sozinho já resolve: a equipe só acrescenta custo e tempo. A tarefa do benchmark é pequena e sequencial (uma funcionalidade e duas correções no mesmo módulo). Ela não mostra o que acontece numa tarefa que o agente sozinho não dá conta ou que se divide em partes independentes. Esse é o caso que falta medir.

4 O que isso muda no NarraTer

[inferido] A conclusão não é "o NarraTer não deve orquestrar". O transporte do NarraTer (o terminal como API universal, a aresta como permissão, a resposta inteira lida do transcript, o histórico gravado pelo backend) é exatamente o que a literatura pede de um orquestrador. A conclusão é sobre a *política*:

1. **O padrão deve ser um agente só.** Montar uma equipe passa a exigir um motivo: duas ou mais subtarefas independentes com arquivos separados, um verificador com contexto limpo ou um contexto que não cabe num agente.
2. **A equipe que mais vale em código é "um escritor + um revisor"**, de preferência de fornecedores diferentes. É onde o NarraTer tem vantagem única: pôr um Codex revisando um Claude não depende de nenhum fornecedor.

3. **"Pronto" tem que ser verificado no ambiente**, rodando os testes ou o comando de aceite, e não declarado pelo agente.
4. **O gerador de orquestração escolhe entre modelos prontos**, entre eles "sozinho", e registra por que montou ou não a equipe.

5 Limites deste documento

- A literatura foi lida, não reproduzida. Conferimos na fonte os números de Kim et al., da Vals.ai, da Cognition (2026) e do estudo de topologia; os demais estão citados com o link para conferir.
- Nossa medição usa uma tarefa só, já no teto de qualidade, e 3 rodadas por braço. Ela mede custo e tempo, não ganho de qualidade.
- A equipe usa Opus, Sonnet e Haiku; o agente sozinho usa só o Opus. Esta é a comparação que importa ("dou a tarefa ao Opus ou ao Opus com uma equipe?"), mas não isola o efeito de cada modelo.
- O braço da equipe é da versão G, e o braço sozinho é da versão atual (H). As duas diferem só no painel Activity, que não muda o protocolo.

Referências

[1] Y. Kim et al. *Towards a Science of Scaling Agent Systems*. arXiv:2512.08296, 2025. https://arxiv.org/abs/2512.08296

[2] D. Tran e D. Kiela. *Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets*. arXiv:2604.02460, 2026. https://arxiv.org/abs/2604.02460

[3] Q. Wang et al. *Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?* arXiv:2402.18272, 2024. https://arxiv.org/abs/2402.18272

[4] H. Zhang et al. *Stop Overvaluing Multi-Agent Debate: We Must Rethink Evaluation and Embrace Model Heterogeneity*. arXiv:2502.08788, 2025. https://arxiv.org/abs/2502.08788

[5] Vals.ai. *Do Agent Teams Pay Off? A Case Study on Vibe Code Bench*, 2026-10-09. https://www.vals.ai/blogs/multi-agent-vibe-code-bench

[6] C. S. Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489

[7] Anthropic. *How we built our multi-agent research system*, 2025. https://www.anthropic.com/engineering/multi-agent-research-system

[8] *An Empirical Study of Coordination Mode as the First-Class Citizen in From-Scratch Multi-Agent Coding*. arXiv:2607.27877, 2026. https://arxiv.org/abs/2607.27877

[9] LangChain. *Benchmarking Multi-Agent Architectures*, 2025. https://www.langchain.com/blog/benchmarking-multi-agent-architectures

[10] W. Yan (Cognition). *Don't Build Multi-Agents*, 2025. https://cognition.com/blog/dont-build-multi-agents

[11] W. Yan (Cognition). *Multi-Agents: What's Actually Working*, 2026-04-22. https://cognition.com/blog/multi-agents-working

[12] M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657, 2025. https://arxiv.org/abs/2503.13657

[13] L. Advani. *From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents*. arXiv:2606.09863, 2026. https://arxiv.org/abs/2606.09863

[14] S. Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.* arXiv:2308.00352, 2023. https://arxiv.org/abs/2308.00352